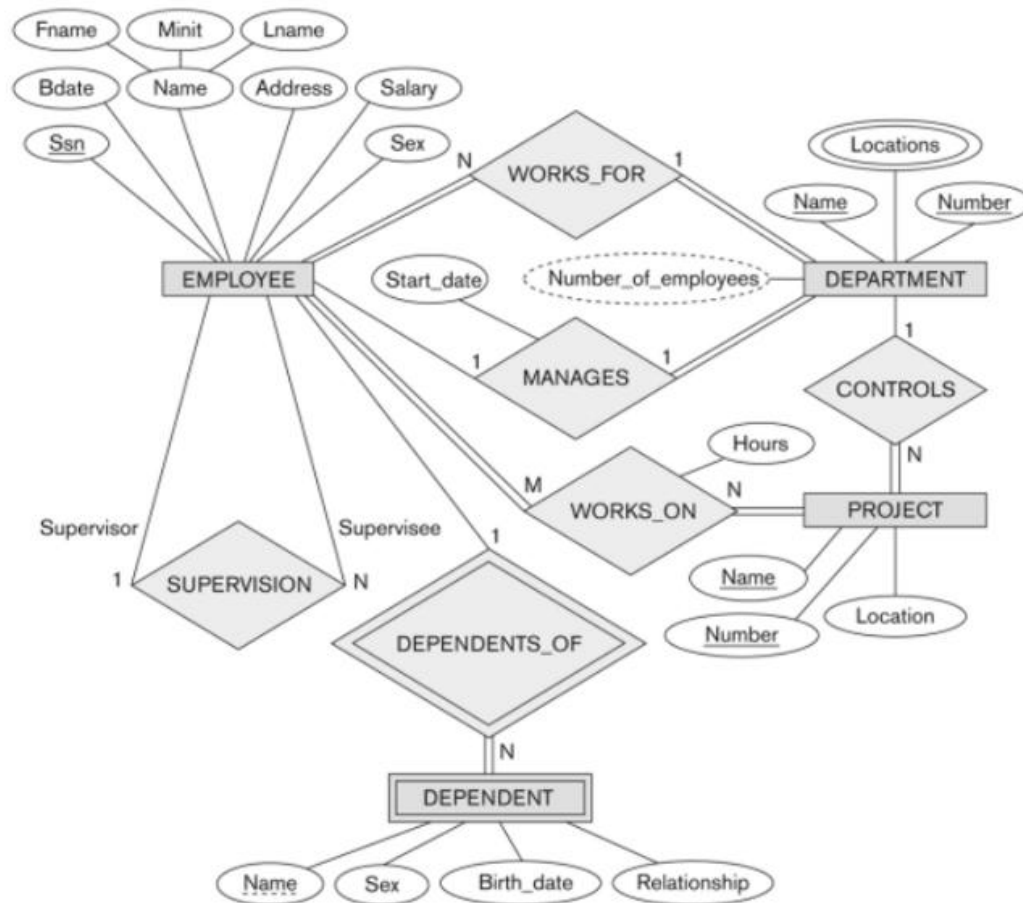


Week 2

Data Modeling

Logical Design (ER to Relational
Mapping)



ER-to-Relational Mapping Algorithm

- Step 1: Mapping of Regular Entity Types
- Step 2: Mapping of Weak Entity Types
- Step 3: Mapping of Binary 1:1 Relation Types
- Step 4: Mapping of Binary 1:N Relationship Types
- Step 5: Mapping of Binary M:N Relationship Types
- Step 6: Mapping of Multivalued attributes

Step 1: Mapping of Regular Entity Types

- For each regular (strong) entity type in the ER schema, create a relation that includes all the simple attributes (includes simple attributes of composite relations)
- Choose one of the key attributes as the primary key .
 - If the chosen key is composite, the set of simple attributes that form it will together form the primary key .
- Example: We create the relations EMPLOYEE, DEPARTMENT, and PROJECT in the relational schema corresponding to the regular entities in the ER diagram. SSN, DNUMBER, and PNUMBER are the primary keys for the relations EMPLOYEE, DEPARTMENT, and PROJECT as shown.

Step 1: Result of Mapping Regular Entity Types

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary
-------	-------	-------	------------	-------	---------	-----	--------

DEPARTMENT

Dname	<u>Dnumber</u>
-------	----------------

PROJECT

Pname	<u>Pnumber</u>	Plocation
-------	----------------	-----------

Step 2: Mapping of Weak Entity Types

- For each weak entity type, create a corresponding relation that includes all the simple attributes.
- Add as a foreign key all of the primary key attribute(s) in the entity corresponding to the owner entity type
- The primary key is the combination of all the primary key attributes from the owner and the partial key of the weak entity, if any

Step 2: Result of Mapping Weak Entity Types

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary
-------	-------	-------	------------	-------	---------	-----	--------

DEPARTMENT

Dname	<u>Dnumber</u>
-------	----------------

PROJECT

Pname	<u>Pnumber</u>	Plocation
-------	----------------	-----------

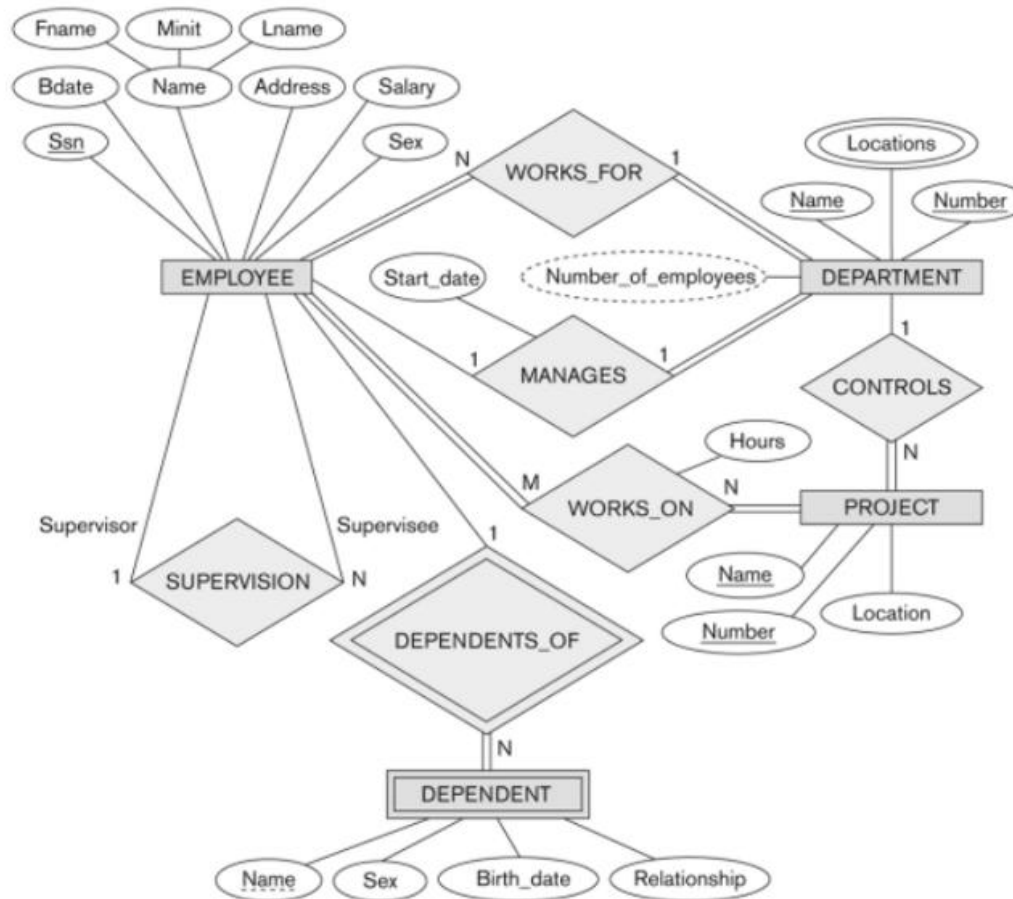
DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

**Dependent_name the partial key with
Essn together will be primary key for Dependent
relation

Step 3: Mapping of Binary 1:1 Relation Types

- Choose one relation that has total participation
- Add to it all the simple attributes of the relationship
- Add as a foreign key the primary key attributes of entity that has *partial participation*
- **Rule:**
- **1:1, one side total participation:** put foreign key in the total side.
- **1:1, both sides total participation:** put foreign key in either side.
- **1:1, both sides partial participation:** make a separate relation for the relationship.



Step 3: Result Mapping of Binary 1:1 Relation Types

Step 3 Result

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary
-------	-------	-------	------------	-------	---------	-----	--------

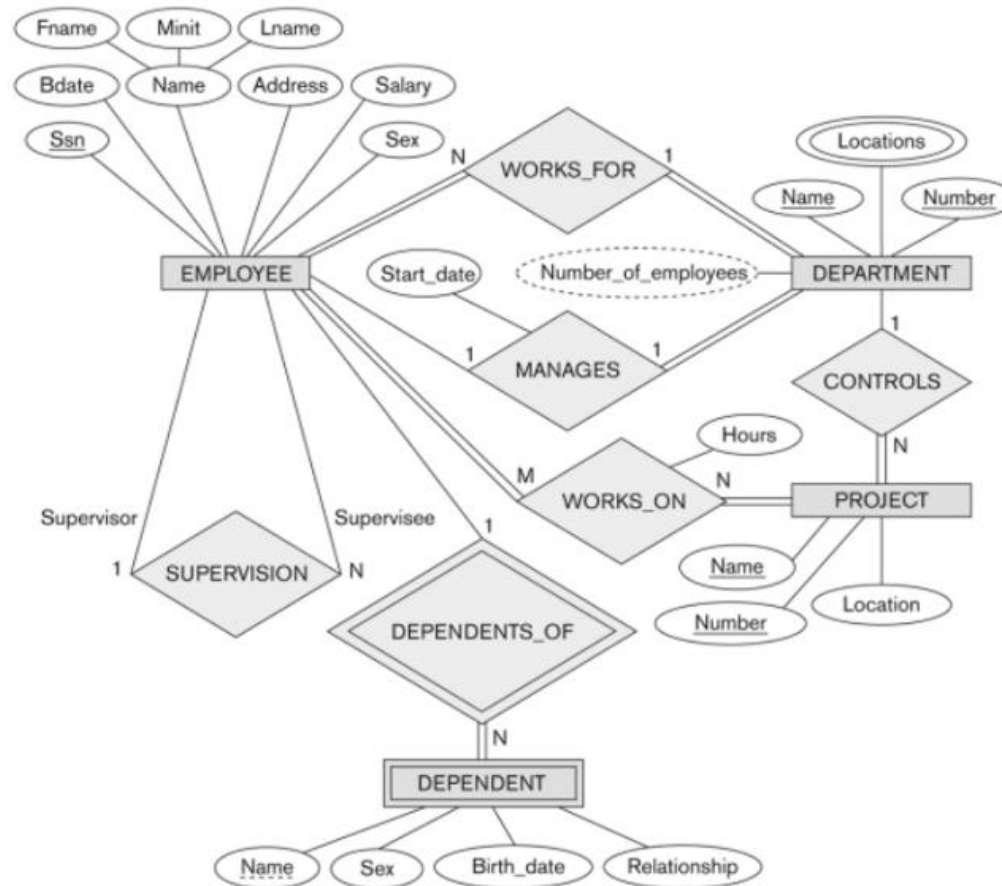
DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

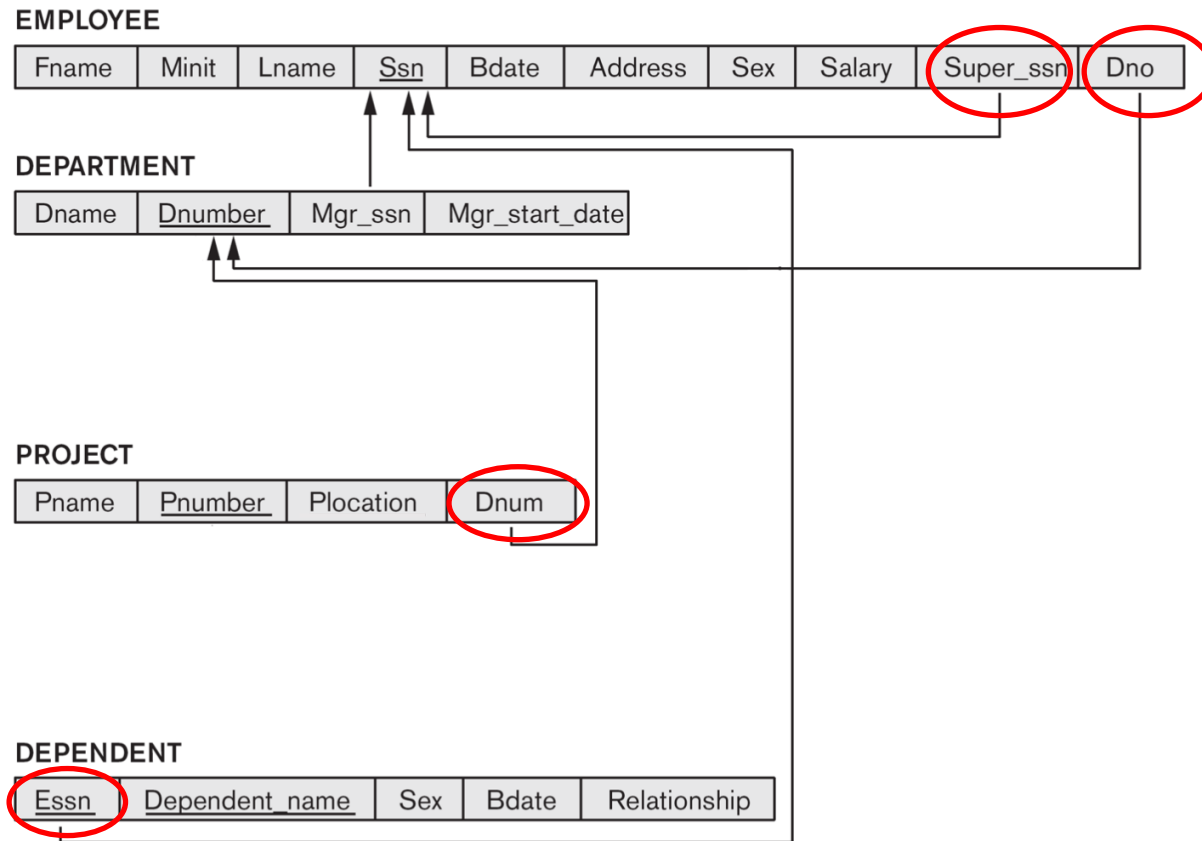


Step 4: Mapping of Binary 1:N Relationship Types

- Choose the relation as the type at the N-side of the relationship.
- Add as a foreign key all of the primary key attribute(s) of 1-side of the relationship.



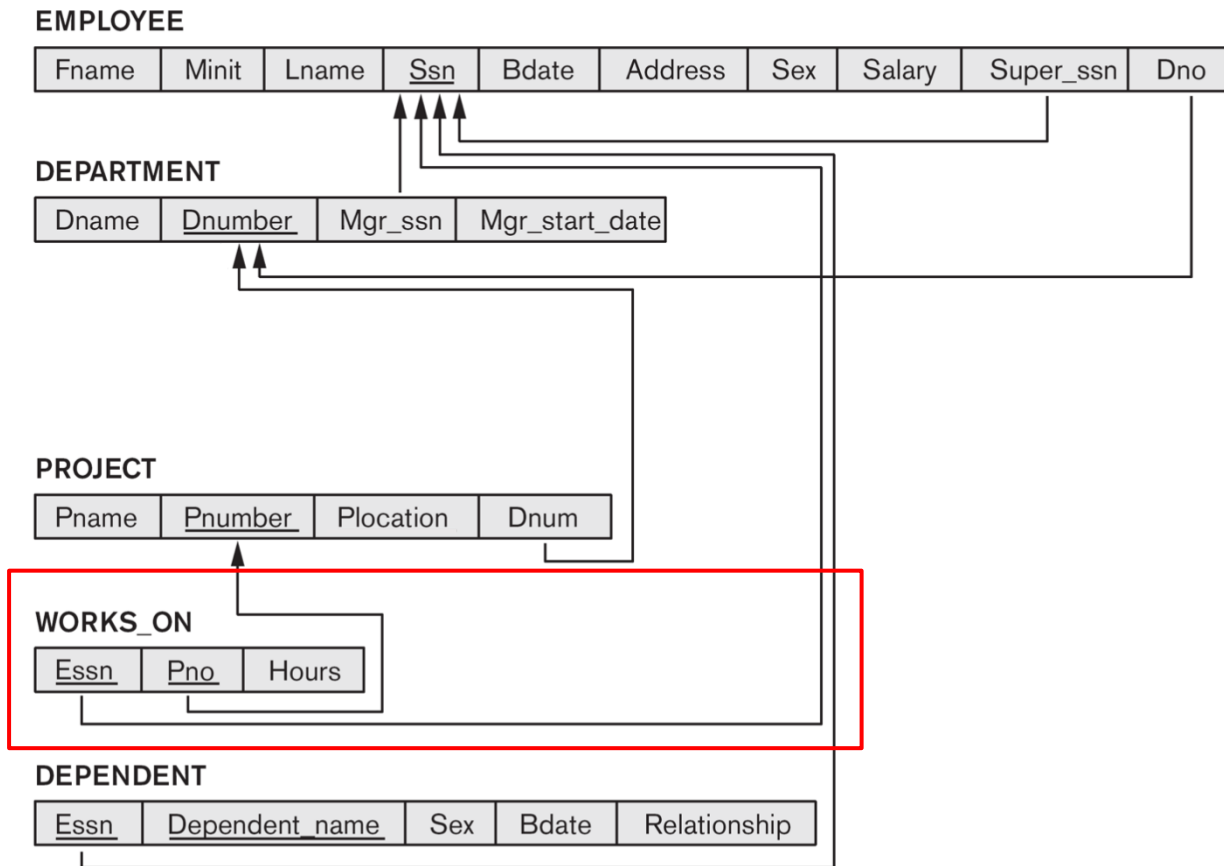
Step 4: Result Mapping of Binary 1:N Relation Types



Step 5: Mapping of Binary M:N Relationship Types

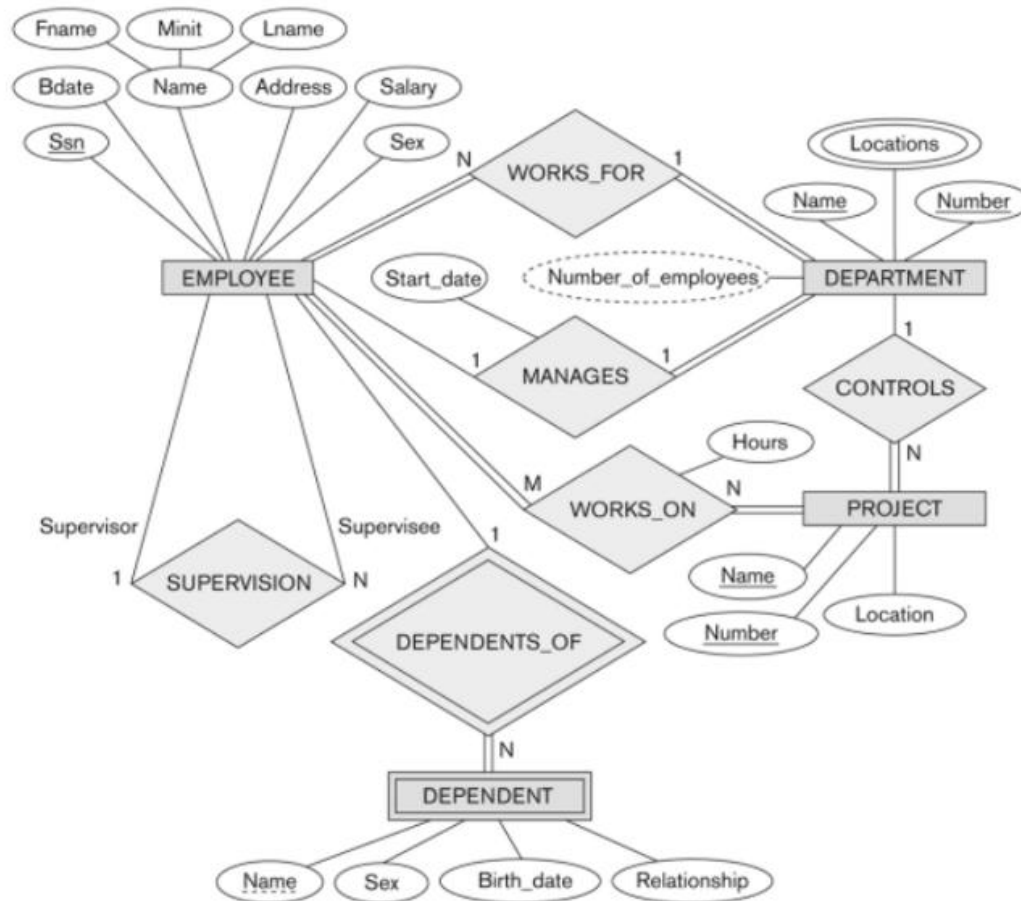
- Create a new relation
- Add as foreign keys the primary keys of both relations
- Add any simple attributes of the M:N relationship

Step 5: Result of Mapping Binary M:N Relationship Types

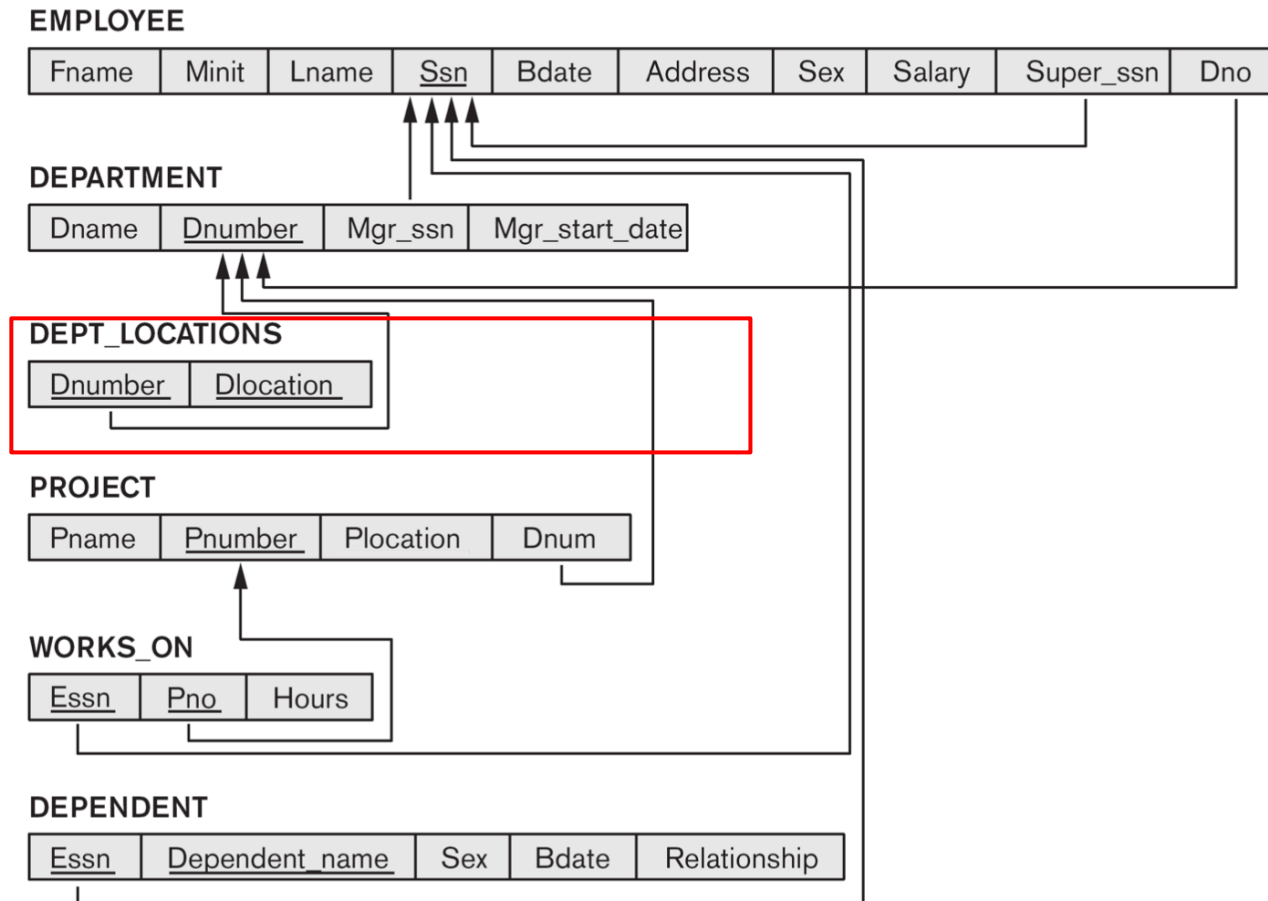


Step 6: Mapping of Multivalued Attributes

- Create a new relation
- Add as foreign keys the primary keys of the corresponding relation
- Add the attribute (if composite, the simple attributes); the combination of foreign key and multivalued attribute forms the primary key



Step 6: Result Mapping of Multivalued Attributes



Resulting Relational Schema

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

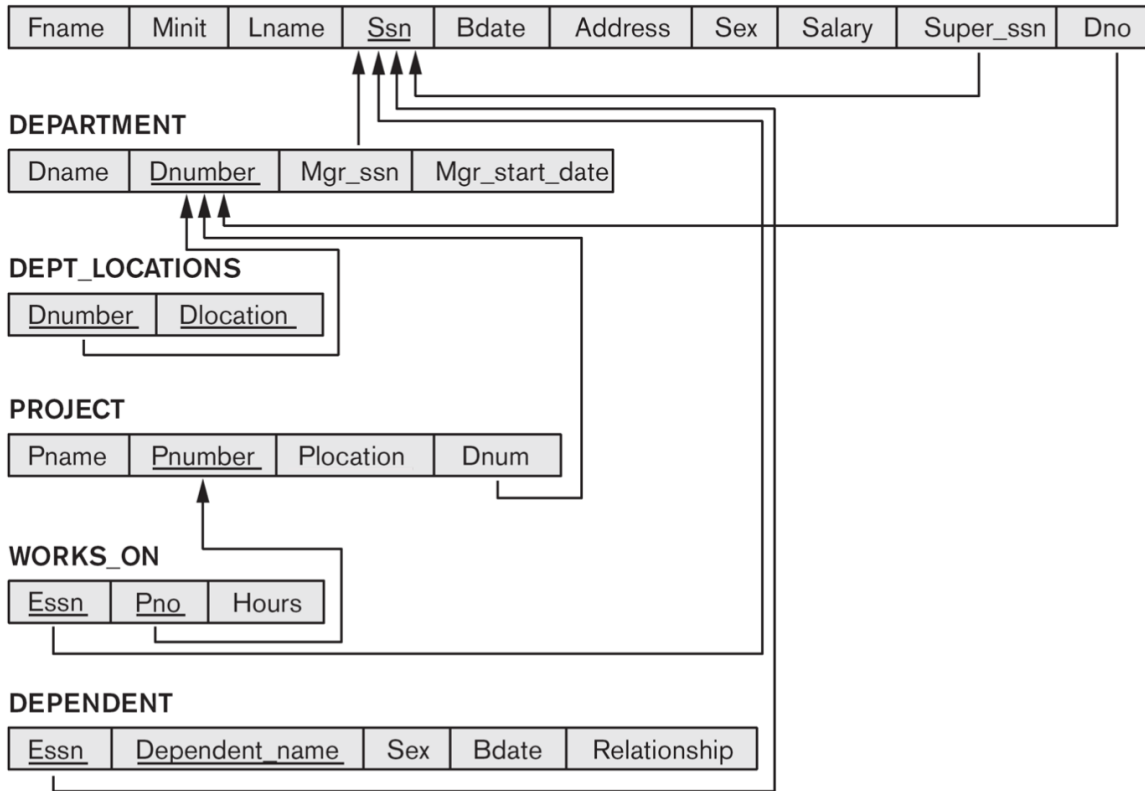
Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------



Question

- Consider the following **ER diagram description**:
- **Entity: Student**
 - Attributes: student_id (primary key), name, email (multivalued), address (composite: street, city, postal_code)
- **Entity: Course**
 - Attributes: course_id (primary key), course_name
- **Relationship: Enrolls** (many-to-many)
 - Between Student and Course
 - Attribute: grade
- **Task:**
- Map this ER model into a set of **relational tables**.
- Identify **primary keys** and **foreign keys** clearly.
- Show how you handle the **multivalued attribute** (email), the **composite attribute** (address), and the **many-to-many relationship** (Enrolls).

Relational Integrity Constraints

Structured Query Language (SQL)part1

Example – A relation *STUDENT*

Relation looks like a table

Diagram illustrating the structure of the **STUDENT** relation:

- Relation Name:** **STUDENT**
- Attributes:** Name, Ssn, Home_phone, Address, Office_phone, Age, Gpa
- Tuples:** Benjamin Bayer, Chung-cha Kim, Dick Davidson, Rohan Panchal, Barbara Benson

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	749-1253	25	3.53
Rohan Panchal	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	NULL	19	3.25

Relational Integrity Constraints

- Constraints are **conditions** that must be true on all valid tuples. These conditions are called **Relational Integrity Constraints**.
- There are three main types of constraints:
 1. **Key** constraints
 2. **Entity integrity** constraints
 3. **Referential integrity** constraints
- These constraints are checked before performing any operation (insert, delete and update) in database. If there is a violation in any of constraints, operation will fail.

1. Key constraints

- Every relation in the database should have at least one set of attributes which defines a tuple *uniquely*. Those set of attributes is called **key**.
- If there are more than one, these are called **candidate keys**.
- **Primary Key**: A candidate key that is chosen to *uniquely* identify each tuple in the relation.
- The primary key attributes are underlined.
- Choose as primary key the smallest of the candidate keys (in terms of size).

2. Entity integrity constraints

- **Entity Integrity:** The primary key attributes of each relation cannot have null values in any tuple of a relation .
 - – No attribute participating in the primary key is allowed to accept null values.
- **Note:** Other attributes of the relation may be similarly constrained to disallow null values, even though they are not members of the primary key.

Key constraints & Entity integrity constraints

- A primary key has two properties:
 1. It should be unique for all tuples.
 2. It can't have NULL values.

Example

STUDENT

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21

STUDENT(Name, SSN, Home_phone, Address, Office_phone, Age, GPA)

Example

STUDENT

Name	<u>Ssn</u>	Home_phone	Address	Office_phone	Age	Gpa
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21

STUDENT(Name, SSN, Home_phone, Address, Office_phone, Age, GPA)

3. Referential integrity constraints

- A constraint involving two relations (the previous constraints involve a single relation).
- **Referential integrity** constraint states that if a relation refers to a key attribute of a different or same relation, then that key element must exist.
- Used to specify a relationship among tuples in two relations: the **referencing relation** and the **referenced relation**.
- Tuples in the **referencing relation** R1 have attributes called **foreign key** attributes(**FK**) that reference the **primary key** attributes (**PK**) of the **referenced relation** R2.
- A value of FK occurs as a value of PK

Example

EMPLOYEE (Referencing relation)

FK

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

PK

(Referenced relation)

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

SQL

- **SQL (Structured Query Language):** is a standard language for storing, manipulating, and retrieving data in relational databases.
- Originally, SQL was called SEQUEL (Structured English QUery Language) and was designed and implemented at IBM Research in 1974 and first released in 1986.
- SQL comes as a package with all major distributions of relational databases management systems (RDBMS).

SQL Syntax

- SQL has its own unique syntax.
- **SQL syntax** used while generating the queries:
 - All SQL commands ends with a **semicolon ;**
 - SQL commands are case insensitive.
 - While writing queries, table name and database name must match with original database and tables created.
 - All SQL commands starts with a **keyword**.
- **SQL Keywords:** Keywords such as **Create, Select, Insert, Drop** etc. have specific functionalities in SQL.

Types of SQL Commands

- **SQL Commands:** Basic SQL statements for storing, retrieving, and manipulating data in a relational database.
- There are several types of SQL commands include:
 - Data Definition Language (DDL)
 - Data Manipulation Language (DML)
 - Data Control Language (DCL)

1. Data Definition Language

- A data definition language (DDL) consists of the SQL commands that can be used to create and modify the **structure of database objects** in a **database**.
- These **database objects** include views, schemas, tables, indexes, etc.
- DDL use:
 - Create (create database, create table)
 - Drop
 - Alter

The CREATE DATABASE Command

- **CREATE DATABASE** command is used to create a new SQL database.
- Syntax:
`CREATE DATABASE databasename;`
- Example: The following SQL statement creates a database called testDB :
`CREATE DATABASE testDB;`

The CREATE TABLE Command

- **CREATE TABLE** Command is used to specify a new table by giving it a name and specifying its attributes and initial constraints.
- The attributes are specified first.
- Each attribute is given a name, a data type to specify its domain of values.

The CREATE TABLE Command

- ▶ **Syntax:**

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype,  
    ....  
);
```

- ▶ **Example:**

```
CREATE TABLE Persons (  
    PersonID int,  
    LastName varchar(255),  
    FirstName varchar(255),  
    Address varchar(255),  
);
```

SQL Data Types

- The data type of an attribute column defines what value the column can hold: integer, character, date and time, and so on.
- Data types might have different names in different DBMS. And even if the name is the same, the size and other details may be different.

MySQL Data Types

I. **String Data Types:** most common types include:

Data type	Description
CHAR (size)	A FIXED length string (can contain letters, numbers, and special characters). The size parameter specifies the column length in characters - can be from 0 to 255. Default is 1
VARCHAR (size)	A VARIABLE length string (can contain letters, numbers, and special characters). The size parameter specifies the maximum column length in characters - can be from 0 to 65,535
MEDIUMTEXT	Holds a string with a maximum length of 16,777,215 characters
LONGTEXT	Holds a string with a maximum length of 4,294,967,295 characters

- ▶ **Be Consistent in the length.** For example use:
VARCHAR (50) for short strings
VARCHAR (225) for medium-length strings

MySQL Data Types

2. Integer Data Types: we use integer to store whole numbers that do not have decimal points. Most common types include:

Data type	Description
TINYINT (size)	A very small integer. Signed range is from -128 to 127. Unsigned range is from 0 to 255. The size parameter specifies the maximum display width (which is 255)
SMALLINT (size)	A small integer. Signed range is from -32768 to 32767. Unsigned range is from 0 to 65535. The size parameter specifies the maximum display width (which is 255)
MEDIUMINT (size)	A medium integer. Signed range is from -8388608 to 8388607. Unsigned range is from 0 to 16777215. The size parameter specifies the maximum display width (which is 255)
INT(size)	A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The size parameter specifies the maximum display width (which is 255)
BIGINT (size)	A large integer. Signed range is from -9223372036854775808 to 9223372036854775807. Unsigned range is from 0 to 18446744073709551615.

MySQL Data Types

3. Fixed-point and floating-points types: used for storing numbers with decimal points. common types include:

Data type	Description
DECIMAL(<i>size</i> , <i>d</i>)	An exact fixed-point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter. The maximum number for <i>size</i> is 65. The maximum number for <i>d</i> is 30. The default value for <i>size</i> is 10. The default value for <i>d</i> is 0.
FLOAT(<i>size</i> , <i>d</i>)	A floating point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter.
DOUBLE(<i>size</i> , <i>d</i>)	A normal-size floating point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter.

► **Example:** DECIMAL (9,2) => 1234567.89

MySQL Data Types

4. Boolean Types: most common types include:

Data type	Description
BOOLEAN	Zero is considered as false, nonzero values are considered as true.

MySQL Data Types

5. Date and Time Data Types: most common types include:

Data type	Description
DATE	A date. Format:YYYY-MM-DD.The supported range is from '1000-01-01' to '9999-12-31'
TIME	A time. Format: hh:mm:ss.The supported range is from '-838:59:59' to '838:59:59'
DATETIME	A date and time combination. Format:YYYY-MM-DD hh:mm:ss.The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'.Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time
TIMESTAMP	A timestamp.TIMESTAMP values are stored as the number of seconds
YEAR	A year in four-digit format.Values allowed in four-digit format: 1901 to 2155, and 0000. MySQL 8.0 does not support year in two-digit format.

A Good Practice

- Use the smallest data type that suits your needs. This would make your database's size smaller and your query executed faster.

SQL Constraints

- SQL constraints are used to specify rules for the data in a table.
- In SQL, we have the following constraints:
- **NOT NULL**: Indicates that a column cannot store NULL value.
- **UNIQUE**: Ensures that each row for a column must have a unique value.
- **PRIMARY KEY**: A combination of a NOT NULL and UNIQUE. Ensures that a column (or combination of two or more columns) have a unique identity which helps to find a particular record in a table more easily and quickly

SQL Constraints

- **FOREIGN KEY:** Ensure the referential integrity of the data in one table to match values in another table.
 - Syntax: **FOREIGN KEY** (column 1, column 2,...) **REFERENCES** table_name (column 1, column 2,...) ;
- **CHECK:** Ensures that the value in a column meets a specific condition.
- **DEFAULT:** Specifies a default value for a column

Example 1

CREATE TABLE DEPARTMENT

(Dname	VARCHAR(15)	NOT NULL,
Dnumber	INT	NOT NULL,
Mgr_ssn	CHAR(9)	NOT NULL,
Mgr_start_date	DATE,	

PRIMARY KEY (Dnumber),
UNIQUE (Dname),
FOREIGN KEY (Mgr_ssn) **REFERENCES** EMPLOYEE(Ssn));

ON DELETE and ON UPDATE

- When you define a **foreign key** in SQL, you tell the database what should happen if the referenced (parent) row is **deleted** or its **primary key updated**.
- FOREIGN KEY (ChildColumn) REFERENCES ParentTable(ParentColumn)
ON DELETE action
ON UPDATE action;

Possible Actions

- **CASCADE**
 - If the parent row is deleted/updated, the change cascades to child rows.
 - Example: If you delete a student, all their enrollments are also deleted.
- **SET NULL**
 - If the parent row is deleted/updated, the child's foreign key is set to NULL.
 - Example: If a department is deleted, students' DeptID becomes NULL.
- **SET DEFAULT**
 - Child's foreign key is set to its **default value** (rarely used).
- **RESTRICT (or NO ACTION)**
 - Prevents the delete/update if matching child rows exist.
 - Example: You cannot delete a department if students are still assigned to it.

Example

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50)  
);  
  
CREATE TABLE Enrollments (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

- If you **delete a student** → all their enrollments are deleted (ON DELETE CASCADE).
- If you **update StudentID** → the change is propagated to Enrollments (ON UPDATE CASCADE).

Common Use Cases

- **ON DELETE CASCADE** → for dependent data (e.g., orders and order items).
- **ON DELETE SET NULL** → for optional relationships (e.g., employee's manager field).
- **ON UPDATE CASCADE** → when primary keys might change (rare in practice).
- **ON DELETE RESTRICT** → when you must preserve parent data if children exist.

Example 2

```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    CHECK (Age>=18)  
);
```

Example 3

```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    City varchar(255) DEFAULT 'Sandnes'  
);
```

The ALTER TABLE Command

- The definition of a table can be changed by using the ALTER command.
- ALTER TABLE actions include:
 1. Adding or dropping a column (attribute)
 2. Changing a column definition;
 3. **General Syntax:** `ALTER TABLE table_name Action column_name datatype;` **ble constraints.**

The ALTER TABLE Command

1. ALTER TABLE - ADD Column

► **Syntax:** `ALTER TABLE table_name
ADD column_name datatype;`

► **Example:** `ALTER TABLE Customers
ADD Email varchar(255);`

2. ALTER TABLE - DROP COLUMN

► **Syntax:** `ALTER TABLE table_name
DROP COLUMN column_name;`

► **Example:** `ALTER TABLE Customers
DROP COLUMN Email;`

The ALTER TABLE Command

3. ALTER TABLE - ALTER/MODIFY COLUMN

- ▶ **Syntax:** `ALTER TABLE` *table_name*
`MODIFY COLUMN` *column_name datatype;*
- ▶ **Example:** `ALTER TABLE` Persons
`MODIFY COLUMN` DateOfBirth year;

The ALTER TABLE Command

4. ALTER TABLE - ADD PRIMARY KEY

- ▶ **Syntax:** `ALTER TABLE table_name
ADD PRIMARY KEY (column_name);`
- ▶ **Example:** `ALTER TABLE student
ADD PRIMARY KEY (student_id);`

5. ALTER TABLE - DROP PRIMARY KEY

- ▶ **Syntax:** `ALTER TABLE table_name
DROP PRIMARY KEY ;`
- ▶ **Example:** `ALTER TABLE student
DROP PRIMARY KEY;`

The ALTER TABLE Command

6. ALTER TABLE - ADD FOREIGN KEY

▶ **Syntax:** `ALTER TABLE` *table_name*
`ADD FOREIGN KEY` (*column_name*) `REFERENCES`
table_name (*column_name*);

▶ **Example:** `ALTER TABLE` Orders
`ADD FOREIGN KEY` (PersonID) `REFERENCES` Persons(PersonID);

7. ALTER TABLE - DROP FOREIGN KEY

▶ **Syntax:** `ALTER TABLE` *table_name*
`DROP FOREIGN KEY` (*column_name*);

▶ **Example:** `ALTER TABLE` Orders
`DROP FOREIGN KEY` (PersonID);

The ALTER TABLE Command

8. ALTER TABLE – ADD a CHECK CONSTRAINT

- ▶ **Syntax:** `ALTER TABLE table_name
ADD CONSTRAINT check_name
CHECK (Age>=18);`
- ▶ **Example:** `ALTER TABLE Persons
ADD CONSTRAINT che123 CHECK (Age>=18);`

9. ALTER TABLE - DROP a CHECK CONSTRAINT

- ▶ **Syntax:** `ALTER TABLE table_name
DROP CONSTRAINT check_name`
- ▶ **Example:** `ALTER TABLE Persons
DROP CONSTRAINT che123;`

The ALTER TABLE Command

10. ALTER TABLE – SET DEFAULT

- ▶ **Syntax:** `ALTER TABLE` *table_name*
`ALTER` *column_name* `SET DEFAULT` 'default value';
- ▶ **Example:** `ALTER TABLE` Persons
`ALTER` City `SET DEFAULT` 'Sandnes';

The DROP DATABASE Command

- **DROP DATABASE** command is used to drop/delete an existing SQL database.

▶ Syntax:

```
DROP DATABASE databasename;
```

▶ **Example:**

- ▶ The following SQL statement drops the existing database "testDB":

```
DROP DATABASE testDB;
```

The DROP TABLE Command

- **DROP TABLE** command is used to drop/delete an existing SQL table.

▶ Syntax:

```
DROP TABLE table_name;
```

▶ **Example:**

- ▶ The following SQL statement drops the existing table "Shippers":

```
DROP TABLE Shippers;
```